Объектно-ориентированное программирование

6 семестр
Лекция 5.

Содержание лекции:

- ASP.NET Core
- Начало работы
- Контроллеры
- Структура HTTP запроса
- REST

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

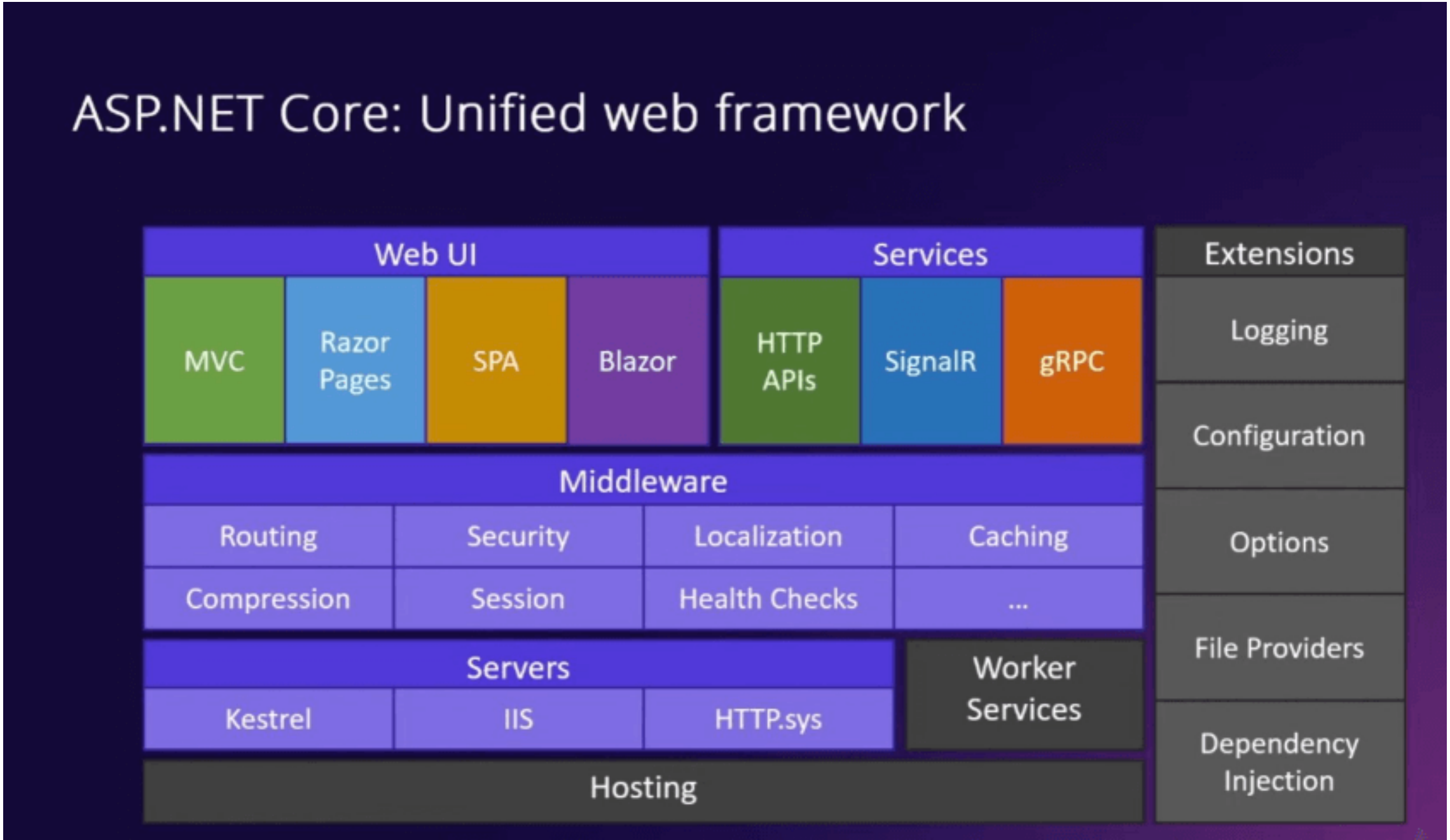
ASP.NET Core. Общие сведения

ASP.NET Core представляет технологию для создания веб-приложений на платформе .NET, развиваемую компанией Microsoft. В качестве языков программирования для разработки приложений на ASP.NET Core используются C# и F#.

История ASP.NET фактически началась с выходом первой версии .NET в начале 2002 года и с тех пор ASP.NET и .NET развивались параллельно: выход новой версии .NET знаменовал выход новой версии ASP.NET, поскольку ASP.NET работает поверх .NET. В то же время изначально ASP.NET была нацелена на работу исключительно в Windows на веб-сервере IIS (хотя благодаря проекту Mono приложения на ASP.NET можно было запускать и на Linux).

Однако 2014 год ознаменовал большие перемены, фактически революцию в развитии платформы: компания Microsoft взяла курс на развитии ASP.NET как кроссплатформенной технологии, которая развивается как opensource-проект. Данное развитие платформы в дальнейшем получило название **ASP.NET Core**, собственно как ее официально именут Microsoft до сих пор. Первый релиз обновленной платформы увидел свет в июне 2016 года. Теперь она стала работать не только на Windows, но и на MacOS и Linux. Она стала более легковесной, модульной, ее стало проще конфигурировать, в общем, она стала больше отвечать требованиям текущего времени.

Архитектура и модели разработки

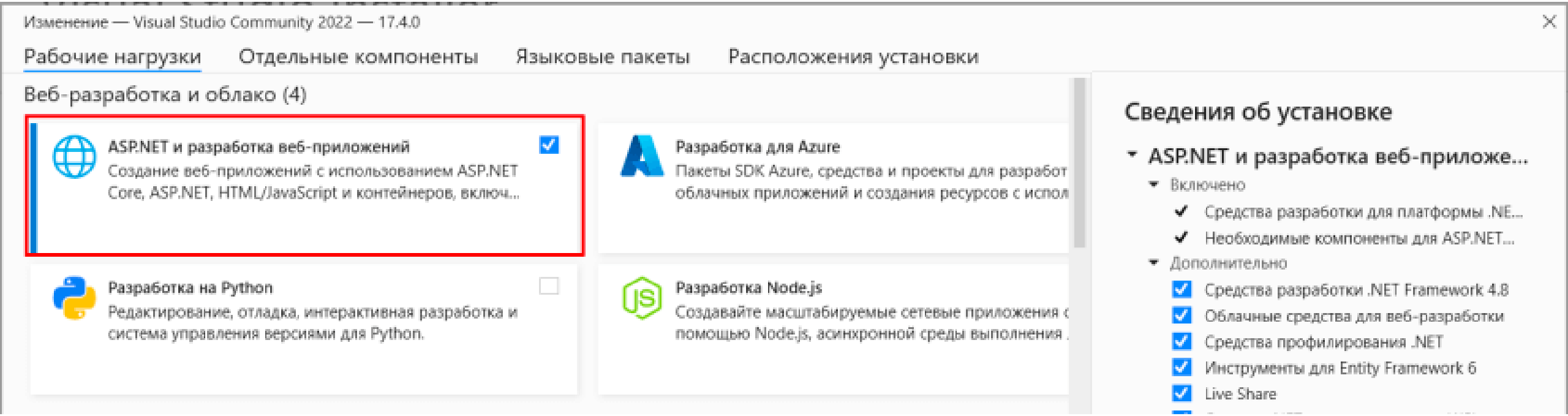


Razor Pages представляет модель, при котором за обаботку запроса отвечают специальные сущности - страницы Razor Pages. Каждую отдельную такую сущность можно ассоциировать с отдельной веб-страницей.

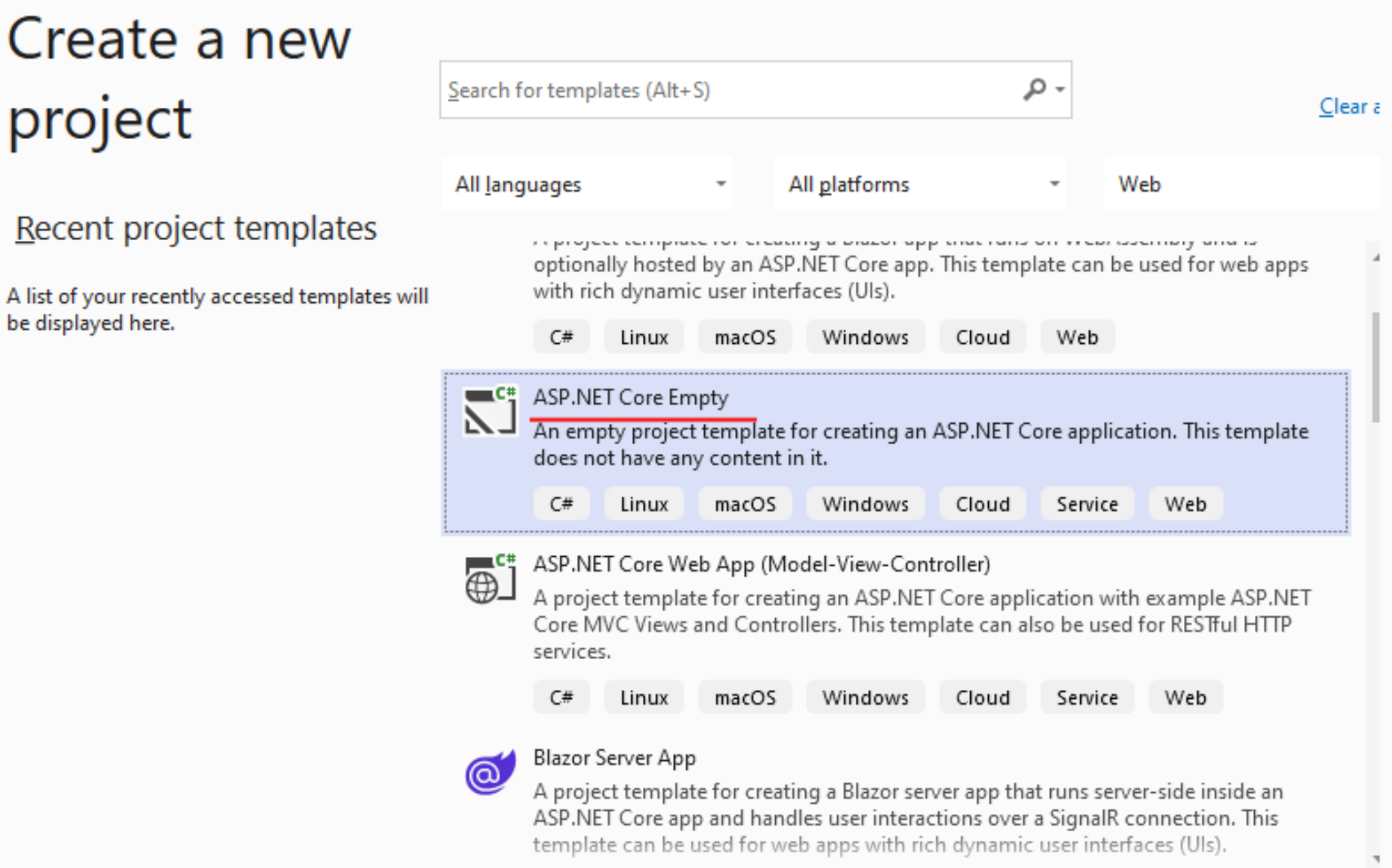
Blazor представляет фреймворк, который позволяет создавать интерактивные приложения как на стороне сервера, так и на стороне клиента и позволяет задействовать на уровне браузера низкоуровневый код WebAssembly.

Начало работы

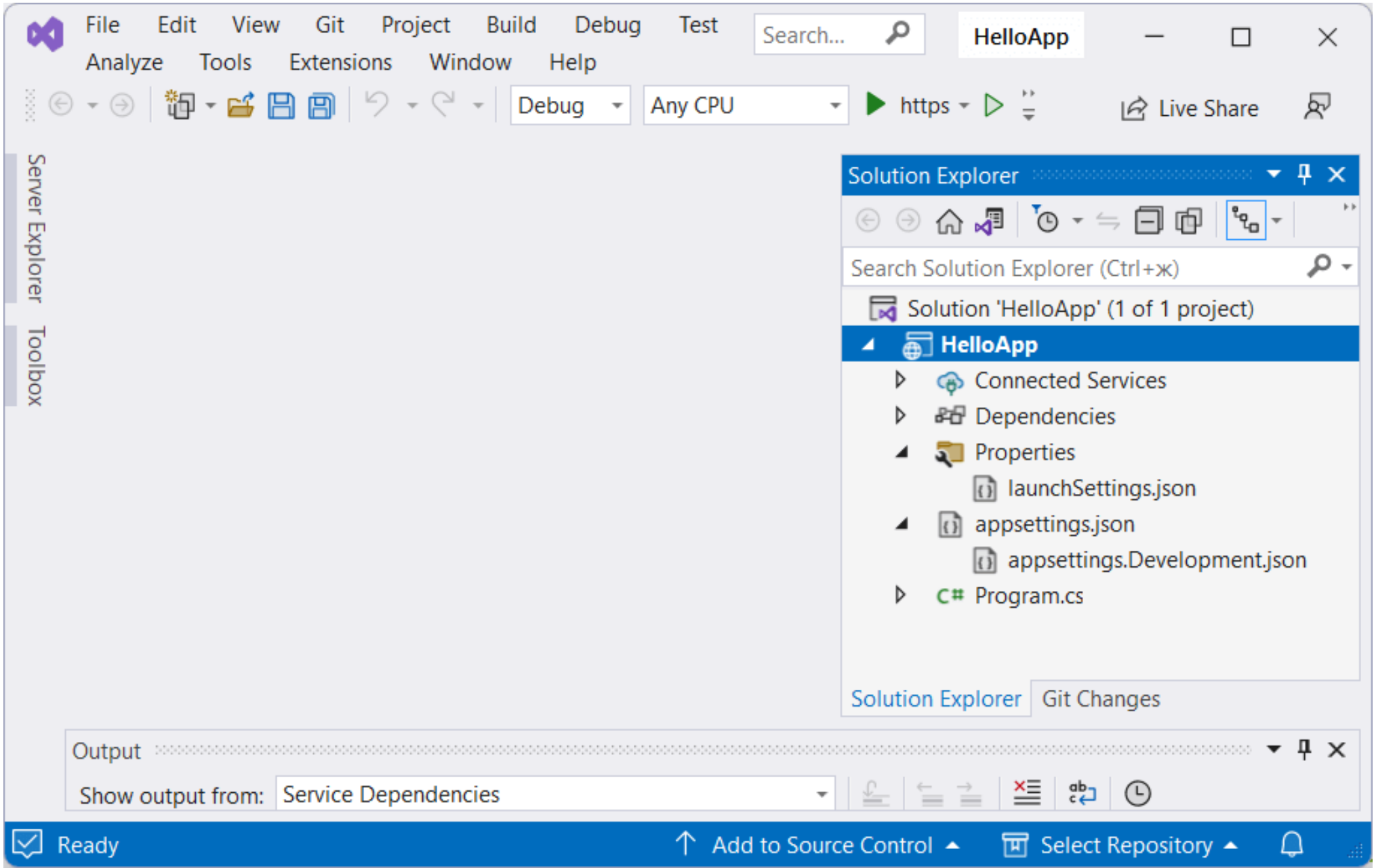
Чтобы добавить в Visual Studio поддержку проектов для ASP.NET Core, в программе установки (Visual Studio Installer) среди рабочих нагрузок нужно выбрать пункт **ASP.NET и разработка веб-приложений.**



Создадим проект **ASP.NET Core Empty**.

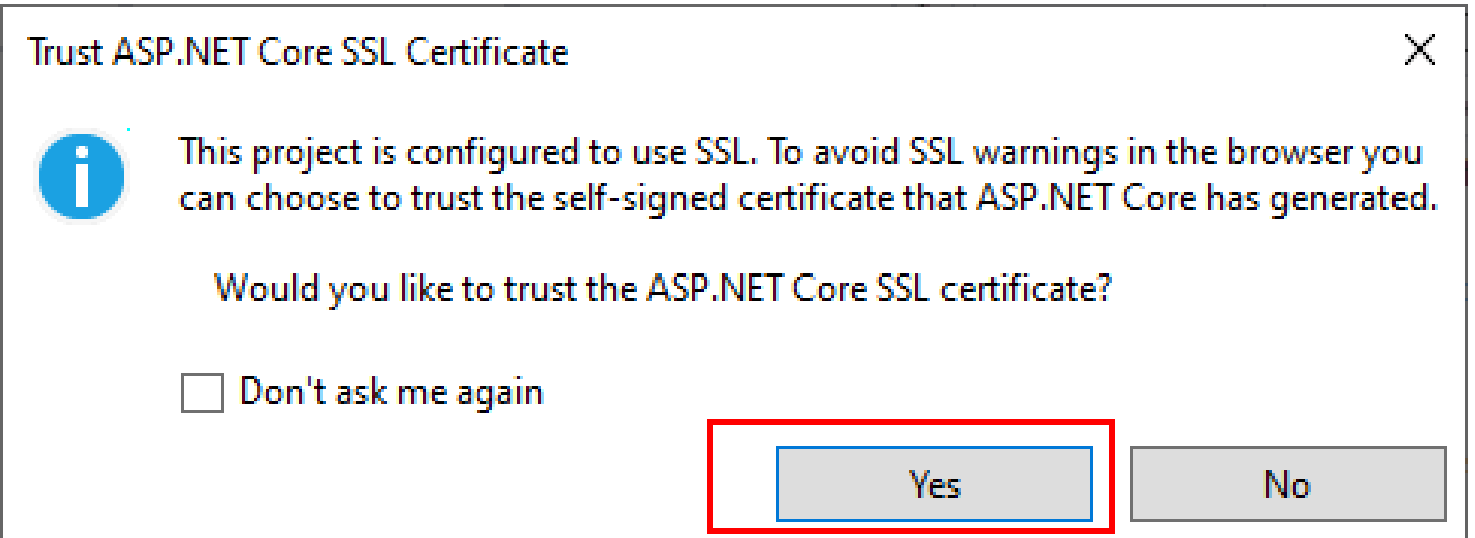


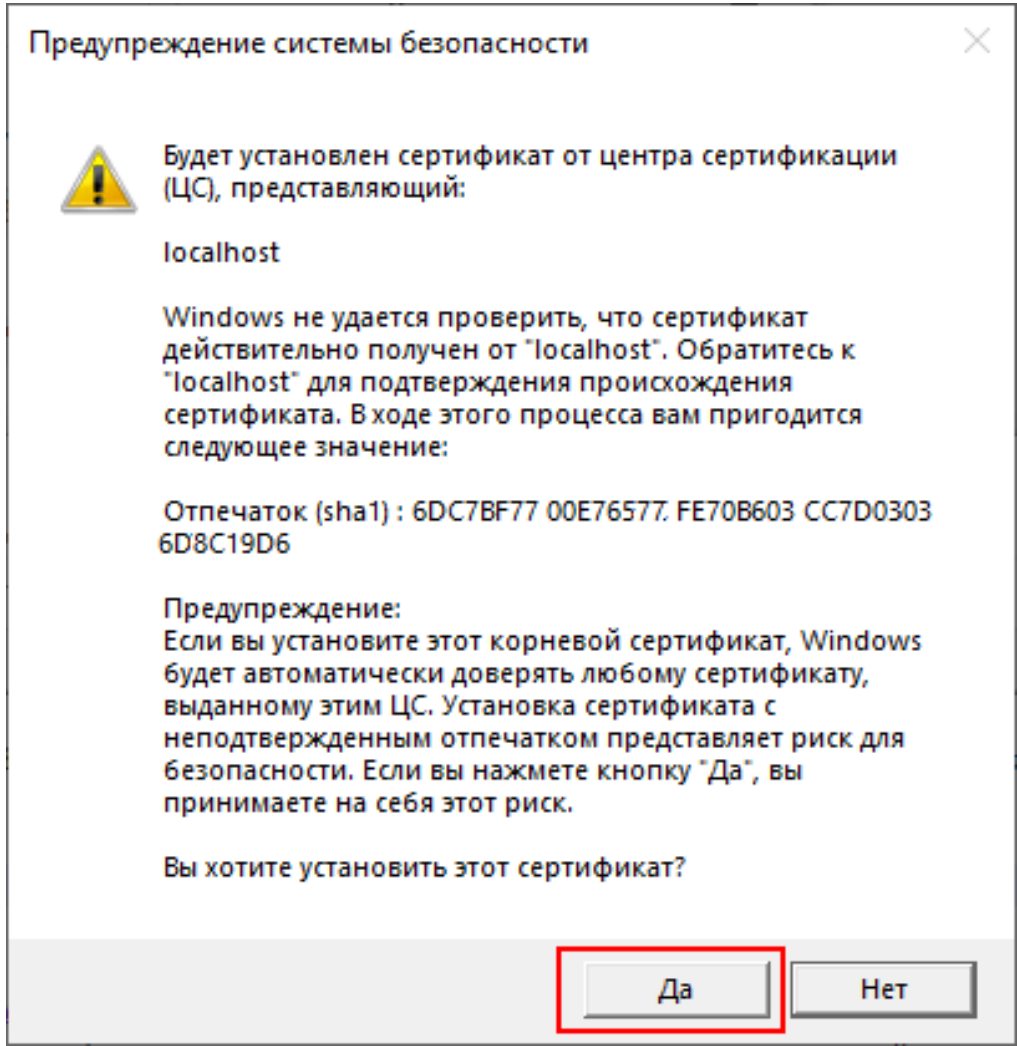
Структура проекта



Запуск проекта

При запуске нам может отобразиться окно, где надо подтвердить доверие для серфиката SSL, а также его установку.





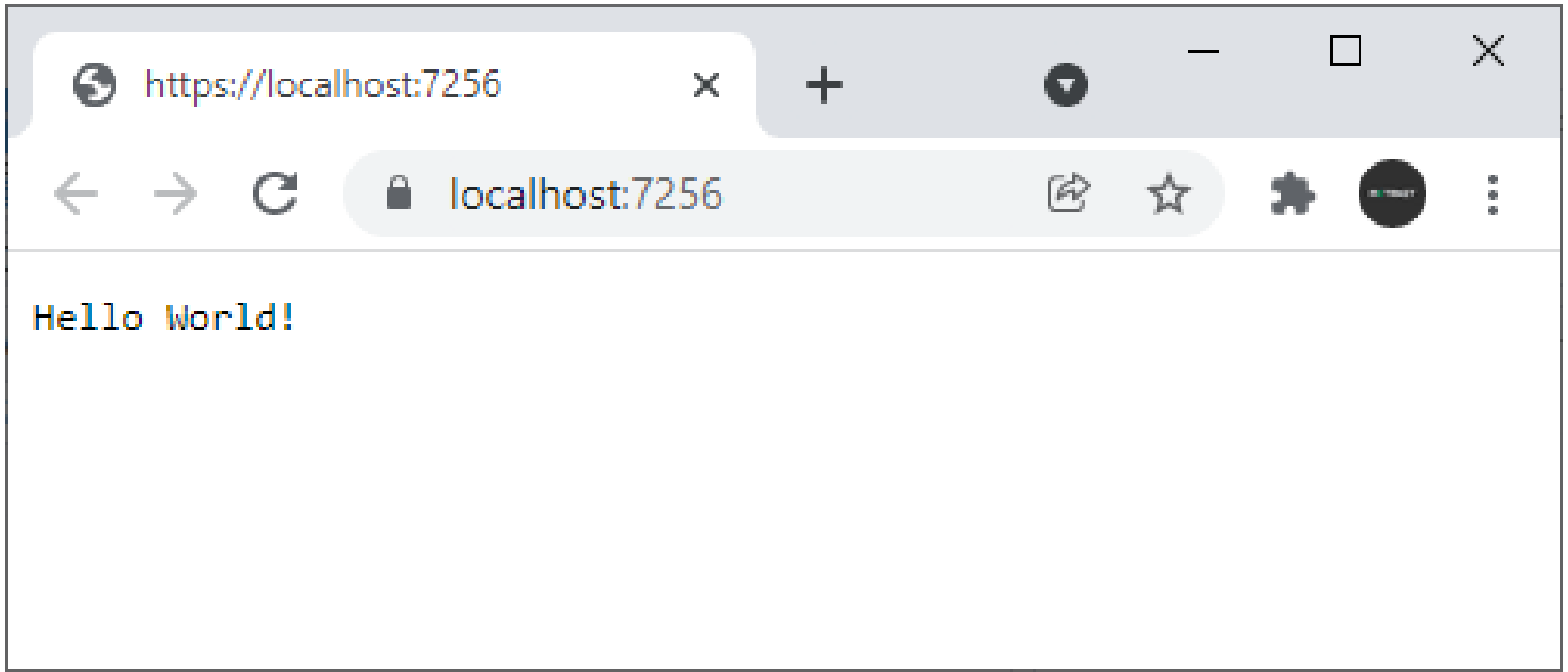
SSL — Secure Sockets Layer или уровень защищенных сокетов это протокол для безопасной связи между браузером и сайтом. Он позволяет передавать данные в зашифрованном виде. Для его реализации и корректной работы нужен SSL-сертификат — цифровой документ, который подтверждает, что ресурс надежен.

Благодаря **SSL** данные между браузером пользователя и сервером сайта шифруются при передаче, и их не может перехватить злоумышленник. Даже если он каким-то образом получит доступ к информации, он не сможет ее прочесть. Поэтому SSL важен для всех сайтов, которые так или иначе работают с персональными данными пользователей, особенно с платежными.

SSL-сертификат — это своеобразная цифровая подпись. Его наличие подтверждает, что пользователь действительно подключается к нужному сайту, владельцу ключа, а не к мошеннику, который подменил адреса и перенаправляет запросы на какой-то другой сервер.

И после подтверждения и установки сертификата отобразиться консоль, где выводися некоторая базовая информация о приложении:

И, кроме того, будет запущен браузер, где мы сможем лицезреть строку "Hello World!" - результат работы кода по умолчанию из файла **Program.cs**:



Контроллеры

Основным элементом в архитектуре ASP.NET Core MVC является **контроллер**. При получении запроса система маршрутизации выбирает для обработки запроса нужный контроллер и передает ему данные запроса. Контроллер обрабатывает эти данные и посылает обратно результат обработки.

При использовании контроллеров существуют некоторые условности. Прежде всего обычно в проекте контроллеры помещаются в каталог **Controllers**. Однако это в принципе необязательно - можно добавлять контроллеры в другие папки или даже в корень проекта. Тем не менее вначале добавим в проект новую папку **Controllers**.

Ключевым элементом контроллера являются его **действия**. Действия контроллера - это публичные методы, которые могут сопоставляться с запросами. Например, возьмем контроллер HomeController и определим в нем метод Index:

```
namespace MvcApp.Controllers
{
    public class HomeController : Controller
    {
        public string Index()
        {
            return "Hello World";
        }
    }
}
```

```
namespace MvcApp.Controllers
{
    public class HomeController : Controller
    {
        public string Index()
        {
            return "Hello METANIT.COM";
        }
    }
}
```

В данном случае метод Index имеет модификатор **public** и поэтому может использоваться для обработки запроса. Данный метод возвращает строку. А это значит, что при обращении к этому действию приложение отправит в ответ пользователю данную строку.

Обращение к действиям контроллера

Сопоставление запроса с контроллером и его действием происходит благодаря системе маршрутизации. И для настройки сопоставления запросов с контроллерами перейдем к файлу **Program.cs** и изменим его следующим образом:

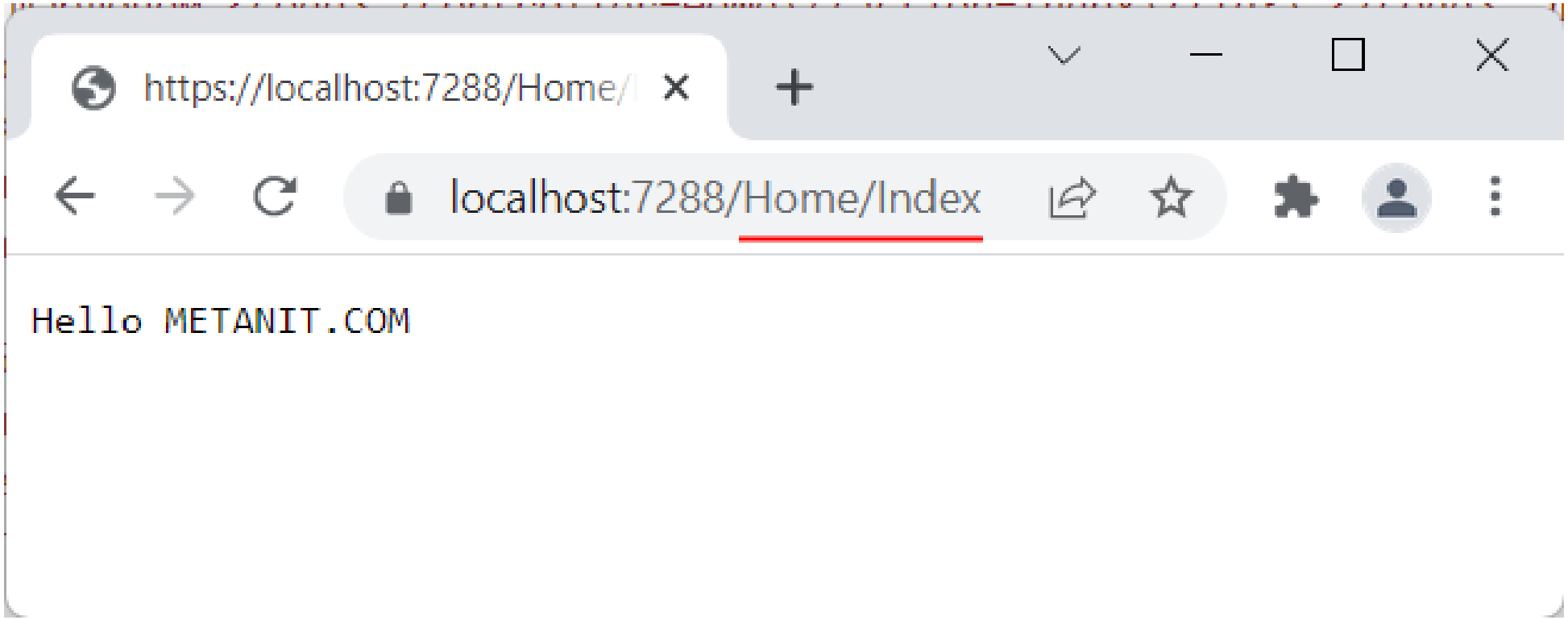
```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();  // добавляем поддержку
контроллеров
var app = builder.Build();
// устанавливаем сопоставление маршрутов с контроллерами
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Метод **app.MapControllerRoute** добавляет один маршрут с именем **default** и шаблоном "{controller=Home}/{action=Index}/{id?}". Данный шаблон устанавливает трехсегментную структуру строки запроса: controller/action/id. То есть в начале идет название контроллера, затем название действия, и далее может идти необязательный параметр id.

Чтобы обратиться контроллеру из браузера, надо набрать в адресной строке **адрес_сайта/Имя_контроллера/Действие_контроллера**. Так, по запросу **адрес_сайта/Home/Index** система маршрутизации по умолчанию вызовет метод Index контроллера HomeController для обработки входящего запроса. Например:



Получение данных через строку запроса

Вместе с запросом приложению могут приходить различные данные. И чтобы получить эти данные, мы можем использовать разные способы. Самым распространенным способом считается применение параметров.

Определение в методах контроллера параметров ничем не отличается от определения параметров в языке C#.

Например, определим в контроллере следующий метод:

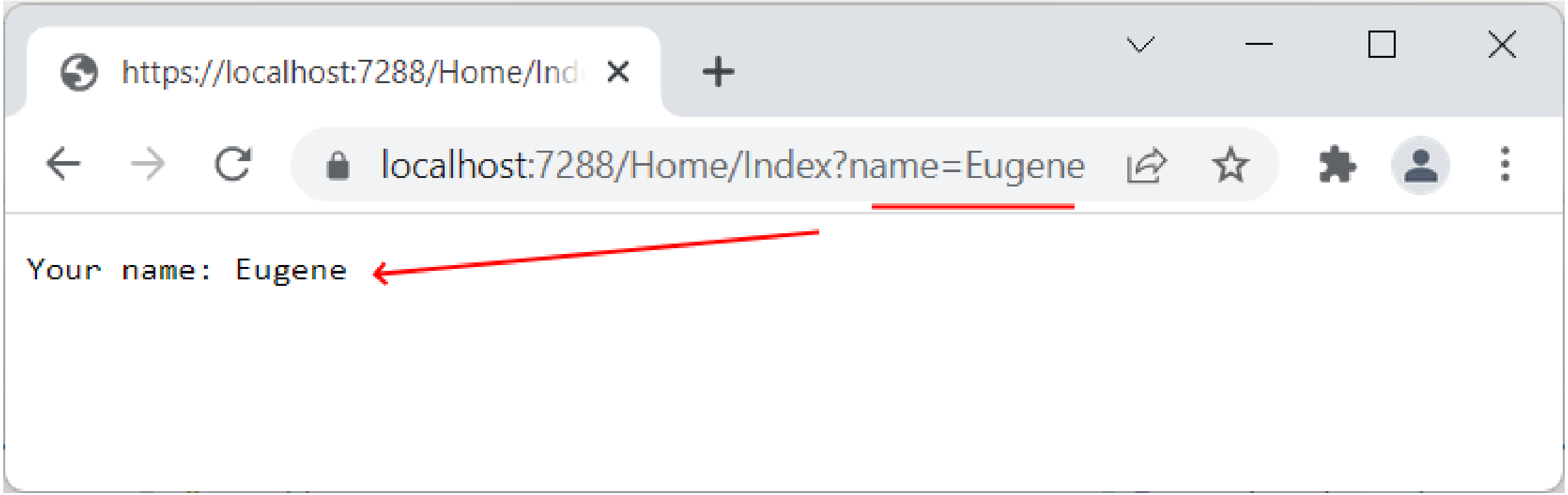
```
namespace MvcApp.Controllers
{
    public class HomeController : Controller
    {
        public string Index(string name) => $"Your name: {name}";
    }
}
```

Передавать значения для параметров можно различными способами. При отправке **GET-запроса** значения можно передать через строку запроса. Строка запроса представляет ту часть адреса, которая идет после знака вопроса **?** и представляет набор параметров, где каждый параметр отделен от другого с помощью амперсанда:

https://localhost:7288/Home/Index?name=Tom&age=37

часть ?name=Tom&age=37 представляет строку запроса, которая содержит два параметра: name и age. Значение параметра name - "Tom", а значение параметра age - 37.

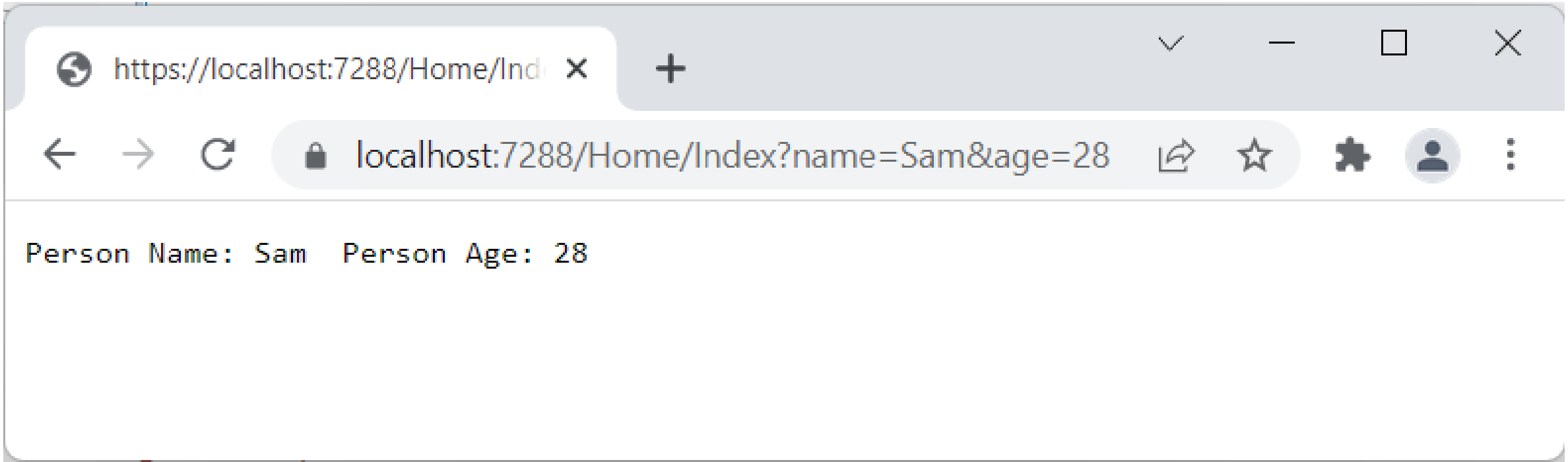
Например, передадим в выше определенный метод Index через строку запроса данные для параметра name:



Передача сложных объектов

```
namespace MvcApp.Controllers
{
    public class HomeController : Controller
    {
        public string Index(Person person)
        {
            return $"Person Name: {person.Name}  Person Age:
{person.Age}";
        }
    }
    public record class Person(string Name, int Age);
}
```

Класс **Person** определяет два свойства: **Name** и **Age**. И в контроллере HomeController метод Index принимает параметр типа Person. В этом случае значения через строку запроса передаются как и в предыдущем случае. При этом параметры строки запроса должны соответствовать по имени свойствам объекта. Регистр названий параметров при этом не учитывается:



Передача массивов сложных объектов

```
public class HomeController : Controller
{
    public string Index(Person[] people)
    {
        string result = "";
        foreach (Person person in people)
        {
            result = $"{result} {person.Name}; ";
        }
        return result ;
    }
}

public record class Person(string Name, int Age);
```

https://localhost:7288/Home/Index?people[0].name=Tom&people[0].age=37
&people[1].name=Bob&people[1].age=41

Также можно опустить название параметра и оставить только индексы:

https://localhost:7288/Home/Index?[0].name=Tom&[0].age=37
&[1].name=Bob&[1].age=41

Структура HTTP запроса

HTTP (англ. Hypertext Transfer Protocol — «протокол передачи гипертекста») — сетевой протокол прикладного уровня, который изначально предназначался для получения с серверов гипертекстовых документов в формате HTML, а с течением времени стал универсальным средством взаимодействия между узлами как Всемирной паутины, так и изолированных веб-инфраструктур.

HTTP - структура

Пример запроса

Метод URL Версия

Стартовая строка (Starting line) POST /login HTTP/1.0

Заголовки (англ. Headers)

```
Host: example.com
Content-Type: application/x-www-form-urlencoded; charset=utf-8
Content-Length: 26
```

Тело сообщения (Body)

```
{
  login=user
  password=qwerty
}
```

Запрос

```
POST    /login    HTTP/1.0

Host: example.com
Content-Type: application/json
Content-Length: 26

{
  login=user
  password=qwerty
}
```

Стартовая строка (Starting line)

Сервер

Ответ

```
HTTP/1.1 200 OK

Content-Type: application/json
Connection: Closed

{
   message: 'success login',
   user : {id: '1', username ... }
}
```

Строка статуса (status line)

Изначально (в начале 90-х) предполагалось, что клиент может хотеть от ресурса только одно — получить его, однако сейчас по протоколу **HTTP** можно создавать посты, редактировать профиль, удалять сообщения и многое другое. И эти действия сложно объединить термином «получение».

Для разграничения действий с ресурсами на уровне **HTTP-методов** и были придуманы следующие варианты:

GET — получение ресурса

POST — создание ресурса

PUT — обновление ресурса

PATCH — обновление фрагмента ресурса

DELETE — удаление ресурса

REST

REST (Representational State Transfer) — это термин был введен в 2000-м году Роем Филдингом (Roy Fielding) — одним из разработчиков протокола HTTP — в качестве названия группы принципов построения веб-приложений.

Вообще REST охватывает более широкую область, нежели HTTP — его можно применять и в других сетях с другими протоколами.

REST описывает принципы взаимодействия клиента и сервера, основанные на понятиях «ресурса» и «глагола» (можно понимать их как подлежащее и сказуемое). В случае HTTP ресурс определяется своим URI, а глагол — это HTTP-метод.

REST предлагает отказаться от использования одинаковых URI для разных ресурсов и использовать разные HTTP-методы для разных действий. То есть веб-приложение, написанное с использованием REST подхода будет удалять ресурс при обращении к нему с HTTP-методом DELETE.

то есть адреса двух разных статей вроде

/index.php?article_id=10 и /index.php?article_id=20 — это не REST-way

Согласно REST адреса будут следующие:

/index.php/articles/10 и /index.php/articles/20

REST дает программистам возможность писать стандартизованные и чуть более красивые веб-приложения, чем раньше. Используя REST, URI для добавления нового юзера будет не /user.php?action=create (метод GET/POST), а просто /user.php (метод строго POST).

Принципы REST API

У RESTful есть 7 принципов написания кода интерфейсов.

Отделение клиента от сервера (Client-Server). Клиент — это пользовательский интерфейс сайта или приложения, например, поисковая строка видеохостинга. В REST API код запросов остается на стороне клиента, а код для доступа к данным — на стороне сервера. Это упрощает организацию API, позволяет легко переносить пользовательский интерфейс на другую платформу и дает возможность лучше масштабировать серверное хранение данных.

Отсутствие записи состояния клиента (Stateless). Сервер не должен хранить информацию о состоянии (проведенных операций) клиента. Каждый запрос от клиента должен содержать только ту информацию, которая нужна для получения данных от сервера.

Кэшируемость (Casheable). В данных запроса должно быть указано, нужно ли делать кэширование данных в Rest API (сохранять в специальном буфере для частых запросов). Если такое указание есть, клиент получит право обращаться к этому буферу при необходимости.

Единообразие интерфейса Rest (Uniform Interface). Все данные должны запрашиваться через один URL-адрес стандартными протоколами, например, HTTP. Это упрощает архитектуру сайта или приложения и делает взаимодействие с сервером понятнее.

Многоуровневость системы (Layered System). В RESTful сервера могут располагаться на разных уровнях, при этом каждый сервер взаимодействует только с ближайшими уровнями и не связан запросами с другими.

Предоставление кода по запросу (Code on Demand). Серверы могут отправлять клиенту код по требованию (например, скрипт для запуска видео). Так общий код приложения или сайта становится сложнее только при необходимости.

Доп. материал REST: https://habr.com/ru/articles/590679/

Статус коды

Это трёхзначное число в ответе, которое характеризуют статус его обработки. Первая цифра обозначает код класса. Всего их пять. Расскажем немного подробнее о каждом и распространённых кодах.

1xx – Informational. С помощью этих кодов состояния описывается сам процесс передачи информации. Как пример, сервер успешно принял запрос, но пока что ещё обрабатывает его.

2xx – Success. Этот класс кодов используется для того, чтобы проинформировать клиента об успешности его запроса.

3xx – Redirection. Такой класс кодов состояния используется в тех случаях, когда для выполнения операции нужно изменить запрос. Чаще всего это связано с неправильным URI ресурса.

4xx – Client Error. Коды состояния, которые относятся к этому классу, используются в случаях, когда во время выполнения запроса произошла ошибка на стороне клиента.

400 Bad Request. Клиент составил своё HTTP-сообщение неправильно.

403 Access Forbidden. К ресурсу необходим другие права доступа.

404 Not Found. Такой код состояния встречается чаще всего и означает, что сервер принял запрос, но по указанному адресу ничего не обнаружил.

5xx – Server Error. Это похожий класс кодов состояния, которые сигнализируют об ошибке. Однако в этот раз ошибка произошла на стороне сервера. Вот несколько вариантов:

500 Internal Server Error. Сервер не смог обработать запрос из-за внутренней ошибки. Например, в коде есть проблемы.

502 Bad Gateway и 504 Gateway Time Out. Иногда сервер выступает в роли промежуточного узла. Если следующий узел вернёт ошибку, сервер в ответе укажет код состояния 502. Если не ответит за отведённое время, то 504.

503 Service Unavailable. Такой код состояния означает, что сейчас на сервере технические неполадки и он не может обработать запрос.

Передача данных через формы в запросе POST

Кроме **GET-запросов** также широко применяются **POST-запросы**. Как правило, такие запросы отправляются с помощью форм на веб-странице. Но основные принципы передачи данных будут теми же, что и в GET-запросах.

Для передачи **POST-запросов** определим в следующий контроллер с двумя методами:

```
public class HomeController : Controller
    {
        [HttpGet]
        public async Task Index()
        {
            string content = @"<form method='post'>
                <label>Name:</label><br />
                <input name='name' /><br />
                <label>Age:</label><br />
                <input type='number' name='age' /><br />
                <input type='submit' value='Send' />
            </form>";
            Response.ContentType = "text/html;charset=utf-8";
            await Response.WriteAsync(content);
        }
        [HttpPost]
        public string Index(string name, int age) => $"{name}: {age}";
    }
```

Первый метод Index имеет атрибут **[HttpGet]**, поэтому данный метод будет обрабатывать только запросы GET. Для упрощения примера в ответ метод будет возвращать html-код с формой ввода (хотя естественно, для формы html можно было бы определить отдельную html-страницу или представление)

Эта форма содержит два поля ввода. Что важно, первое поле имеет имя "name", которое задается с помощью атрибута "name":

```
<input name='name' />
```

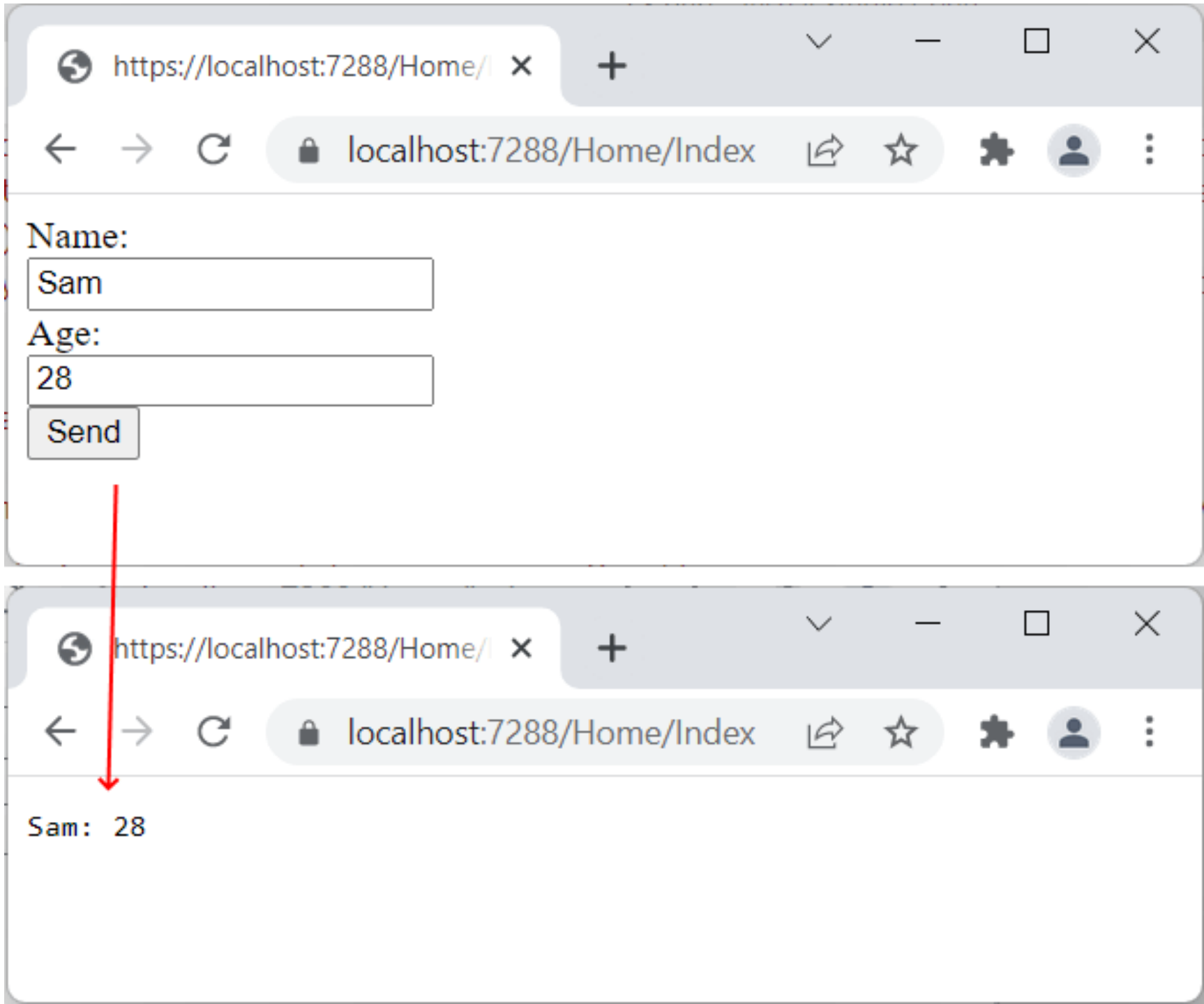
Таким образом, при обращении к методу пользователь увидит в браузере форму ввода. При нажатии на кнопку Send введенные данные будут отправляться на сервер. Поскольку у элемента <form> не задан атрибут action, который устанавливает адрес, то введенные данные отправляются на тот же адрес (то есть по сути методу с тем же именем - методу Index). Но поскольку у формы установлен атрибут method='post', то данные будут отправлять в запросе типа POST. А запросы данного типа обрабатывает второй метод Index:

```
[HttpPost]
public string Index(string name, int age) => $"{name}: {age}";
```

Чтобы система могла связать параметры метода и данные формы, необходимо, чтобы атрибуты name у полей формы соответствовали названиям параметров. То есть в данном случае параметры метода Index называются так же, как и поля формы - name и age.

```
[HttpPost]
public string Index(string name, int age) => $"{name}: {age}";
```

Результат работы программы



Передача зависимостей в контроллер

Как и любой класс, контроллер может получать сервисы приложения через механизм **dependency injection**. В контроллере это можно делать следующими способами:

- Через конструктор
- Через параметр метода, к которому применяется атрибут **FromServices**
- Через свойство **HttpContext.RequestServices**

В данном случае определен интерфейс **ITimeService** и его реализация **SimpleTimeService**. И в приложении происходит регистрация сервиса **ITimeService**.

```
public interface ITimeService
{
    string Time { get; }
}

public class SimpleTimeService: ITimeService
{
    public string Time => DateTime.Now.ToString("hh:mm:ss");
}
```

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();  // добавляем поддержку
контроллеров
builder.Services.AddTransient<ITimeService, SimpleTimeService>();
// добавляем сервис ITimeService

var app = builder.Build();

// устанавливаем сопоставление маршрутов с контроллерами
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Передача через конструктор

Когда приходит запрос к контроллеру, инфраструктура MVC вызывает провайдер сервисов для создания объекта HomeController. Провайдер сервисов проверят конструктор класса HomeController на наличие зависимостей. Затем создает объекты для всех используемых зависимостей и передает их в конструкторо HomeController для создания объекта контроллера, который затем обрабатывает запрос.

Например, получим зависимость в конструкторе контроллера:

```
public class HomeController : Controller
{
    readonly ITimeService timeService;

    public HomeController(ITimeService timeServ)
    {
        timeService = timeServ;
    }

    public string Index() => timeService.Time;
}
```

В данном случае процесс установки зависимостей будет выглядеть следующим образом:

1. Приложение получает запрос к методу контроллера HomeController
2. Фреймворк MVC обращается к провайдеру сервисов для создания объекта контроллера HomeController
3. Провайдер сервисов смотрит на конструктор класса HomeController и видит, что там имеется зависимость от интерфейса ITimeService
4. Провайдер сервисов среди зарегистрированных зависимостей ищет класс, который представляет реализацию интерфейса ITimeService
5. Если нужная зависимость найдена, то провайдер сервисов создает объект класса, который реализует интерфейс ITimeService

6. Затем провайдер сервисов создает объект HomeController, передавая в его конструктор ранее созданную реализацию ITimeService
7. В конце провайдер сервисов возвращает созданный объект HomeController инфраструктуре MVC, которая использует контроллер для обработки запроса

Передача зависимостей в методы. FromServices

Иногда зависимость используется только в одном методе. И в этом случае нет необходимости передавать ее в контроллер, поскольку она напрямую может быть внедрена в сам метод, который ее использует. Для передачи зависимости в метод применяется атрибут **[FromServices]**:

```
public string Index([FromServices] ITimeService timeService)
{
    return timeService.Time;
}
```